Name: Madesh M
Role: Junior ML Engineer @ Biz-Tech Analytics
Mail id: madesh6554@gmail.com
Doc Link: Part_2_Answers.pdf

Part 2: Reasoning-Based Answers

Q1: Choosing the Right Approach

I would go with object detection because here we need to check whether the label is present or missing on a specific part of the product, not just classify the whole image. Detection helps to both locate and identify the label area, which is important in an assembly line. If I use classification, it may not work well because products look similar except for the label. As a fallback, I would try classification on cropped regions or switch to segmentation if the label is very small or unclear. Segmentation can help in more precise detection when boundaries are important.

Q2: Debugging a Poorly Performing Model

First, I would check whether the training data and new factory data are similar or not, like lighting, angle, and background. Then I would visualize predictions on new images to see where the model is failing. Next, I would check annotation quality because wrong labels can affect performance. I would also test the model on validation data and compare results. Finally, I would try adding more diverse data or applying augmentation to improve generalization.

Q3: Accuracy vs Real Risk

I don't think accuracy is the right metric here because even with 98% accuracy, missing defective products is risky in real-world scenarios. In this case, recall is more important, especially for detecting defective items. Missing even a few defects can cause serious issues in production. So I would focus more on recall and also check precision-recall balance. Metrics like F1-score and confusion matrix will give better understanding of model performance.

Q4: Annotation Edge Cases

I think we should keep blurry or partially visible objects because they represent real-world conditions. If we remove them, the model may not perform well in actual scenarios. But at the same time, we need to label them carefully to avoid confusing the model. The trade-off is between clean data and realistic data. Keeping such cases improves robustness, but too many low-quality images can reduce accuracy.

Part 1: Gloved vs Ungloved Hand Detection

Project Description

This project focuses on building a safety compliance system to detect whether a person is wearing gloves or not using computer vision techniques. The model is trained using a custom dataset consisting of both publicly available data and manually collected and labeled images.

I used YOLOv8 for object detection due to its speed and efficiency in real-time scenarios. The system takes images as input and detects hands with bounding boxes, classifying them as either gloved or bare hands.

The output includes annotated images along with structured JSON logs for each detection, making it suitable for real-world monitoring applications.

Dataset

The dataset used in this project is a hybrid combination of publicly available datasets and manually collected images.

- ~1500 images from existing datasets
- ~200 manually collected and labeled images
- Total ~2000 images

All images were annotated using Roboflow with bounding box annotations for:

- gloved_hand
- bare_hand

Model Details

- Model: YOLOv8 (Pretrained + Fine-tuned)
- Training Epochs: 30
- Image Size: 640x640

The model was trained on a CPU environment and achieved strong performance on validation data.

Model Performance

- Precision: ~92%
- Recall: ~83%
- mAP@50: ~91%

These results show that the model performs well in detecting both gloved and bare hands in different conditions.

Features Implemented

- Custom dataset creation and labeling
- YOLOv8 model training
- Detection script for inference
- Bounding box visualization
- JSON logging for detections
- Sample output generation

Live Demo

👉 https://gloved-vs-ungloved-hand-detection-m58nfzmdrlkzjxzktms8fm.streamlit.app/

GitHub Repository

👉 https://github.com/madesh6554/Gloved-vs-Ungloved-Hand-Detection

How to Run

1. Install dependencies
2. Run detection script
3. Provide input images
4. Get output images and JSON logs